LLM: как устроены и как с ними работать

01 Токены и контекстное окно

`При` `вет` `,` `как` `де` `ла` `?` 7 токенов

`Hello` `,` `how` `are` `you` `?` 6 токенов · условный токенизатор

Токен — кусок текста по словарю токенизатора: слово, часть слова, знак. В английском ≈ 4 символа на токен, русскому на ту же мысль обычно нужно больше. В токенах считают **цену, лимиты и окно**. Токен LLM ≠ JWT-токен.

◂ вход: промпт — **контекстное окно** — общий бюджет на один вызов — выход ▸

system	история диалога	найденные документы (RAG)	вопрос	**резерв под ответ**	свободно

Не влезло → ошибка API, обрезка старой истории или **оборванный ответ** (`finish_reason = "length"`). Длинный контекст дорогой, а детали в середине модель замечает хуже.

02 Как модель пишет текст — и почему галлюцинирует

Модель считает **вероятности следующего токена**, выбирает один, дописывает и повторяет:

«Столица Франции — »

Париж	0.91
город	0.04
это	0.02
Лион	0.01

↻ выбранный токен дописывается к тексту → снова предсказание, пока не «стоп» или лимит

Почему галлюцинирует

- ищет **правдоподобное**, факты не проверяет — базы фактов нет
- не знает ваших и свежих данных (дата среза)
- «не знаю» — не самый вероятный ответ
- случайность выбора, мутный контекст

Как снизить

- RAG: ответ по источнику со ссылками
- разрешить ответ «не знаю»
- tool calling для живых данных
- низкая temperature, проверка ответа

03 temperature, top-p, top-k



T = 0.2 острый пик — **T = 1** как обучили — **T = 1.5** почти ровно

temperature меняет «остроту» распределения. Низкая — стабильно: факты, код, JSON. Высокая — разнообразно, но больше ошибок. **T = 0 ≠ без ошибок.**



top-k = 3 только k лучших — **top-p = 0.9** набрать сумму до p

Отсекают маловероятные токены. Крутят обычно что-то одно.

04 Роли в LLM API

`SYSTEM` Ты — поддержка магазина. Отвечай кратко.

`USER` Где мой заказ 123?

`ASSISTANT` → вызов `get_order(123)`

`TOOL` {"status": "в пути"}

`ASSISTANT` Заказ в пути.

system — правила, роль и формат от разработчика, приоритет выше. **user** — запрос в этих рамках.

API **не помнит** историю: шлём её в каждом запросе → токенов всё больше. System — не защита от prompt injection: права проверяет код.

05 Few-shot и chain of thought

Zero-shot — без примеров:
`Тональность: «Доставили за час!»`

Few-shot — 2–5 примеров «вход → выход» задают формат и границы:
«Курьер опоздал» → негатив
«Всё пришло целым» → позитив
«Доставили за час!» → ?

Chain of thought — «рассуждай по шагам, потом ответ». Точнее на многошаговых задачах, но дороже и медленнее.

Reasoning-модели рассуждают сами; рассуждение отделяют от ответа.

06 Стриминг ответа



без стриминга — со стримингом

↑ ~0.4 с (TTFT) — весь ответ ~8 с

```
data: {"delta": "Зак"}
data: {"delta": "аз в пути"}
data: [DONE]      # упрощённо
```

Время то же, но **первые слова сразу**. Куски идут по **SSE** (`stream=True`). Для чата и длинных ответов; JSON разбирают, когда собран целиком. Обрыв посередине надо обработать.

07 Гарантированно валидный JSON

1. Просьба в промпте — гарантии нет

2. JSON mode — синтаксис валиден, поля — нет

3. Structured outputs — JSON-схема ограничивает саму генерацию

Ответ temperature 0 → **finish_reason** "stop", не "length" → **Pydantic** форма + бизнес-правила → **✓ в код** разбор, роутинг

```
class Ticket(BaseModel):
    category: Literal["оплата", "доставка", "другое"]
    priority: Literal["low", "high"]
    summary: str = Field(max_length=200)
```

✗ ошибка → повтор с текстом ошибки валидации, **не больше N раз** → фолбэк (категория «другое», ручной разбор). Повтор — страховка, гарантию даёт схема.

08 Как снизить стоимость

(вход × цена + выход × цена) × запросы

выходные токены обычно в разы дороже

- **модель по задаче**, каскад: дешёвая → сильная
- **меньше входа:** RAG вместо всей базы, сжатие истории, короткий system
- **меньше выхода:** `max_tokens`, краткий формат
- **кэш** ответов (и семантический), кэш промпта у провайдера
- **Batch API** для несрочного
- **учёт и лимиты** токенов, следить за ретраями и циклами агентов

09 Открытые и закрытые модели

	ЗАКРЫТЫЕ (API) GPT, CLAUDE, GEMINI	ОТКРЫТЫЕ ВЕСА LLAMA, QWEN, DEEPSEEK
Качество	обычно вершина рейтингов	догоняют; для узких задач хватает
Цена	за токены, старт без вложений	GPU; выгодно при большом объёме
Данные	уходят провайдеру	не покидают твой контур
Контроль	модель могут обновить или снять	версия своя, дообучай как хочешь
Запуск	ничего поднимать не надо	Ollama, vLLM, облачный хостинг

Прототип и сложные задачи → API. Чувствительные данные → открытая в своём контуре. Модель — за своим адаптером, чтобы менять провайдера.